

Zarqa University
Faculty of Information Technology
Software Engineering Department



Composite Design Pattern

Subject: Patterns Programming

Presented by:

Saja Fateh, (202402627)

Shahed Mahmoud, (202430155)

Ghofran Al-Shareef, (202430081)

Malak Saleh, (202302115)

Supervised by:

Dr. Issam Jebreen

Contents

Introduction To Composite Design Pattern	2
1.1 Overview of Composite design patterns:.....	2
1.2 Intent of the pattern:	2
Problem Analysis & Trade-offs.....	3
2.1. Issues in object creation:	3
2.1.1. Poor object creation design.....	3
2.1.2. Constructors with many parameters.....	3
2.1.3. Inconsistent object state:	4
2.1.4. Lack of flexibility	4
2.2. Why traditional approaches fail	4
2.3. Possible design patterns to solve these issues	5
2.3.1. Composite Design Pattern	5
2.3.2. Flyweight Design Pattern	5
Pattern Selection & Justification	5
3.1. Why Composite Design Pattern is Needed	5
3.2. Real-World Analogy	6
3.3. When to Use the Composite Pattern	6
Case Study : Online Shopping Delivery System.....	6
4.1. Problem in Online Shopping Delivery System	6
4.2. Applying Composite Pattern as a Solution	7
Pattern Implementation	8
5.1. Code without Composite	8
5.2. Code with Composite	12
5.3. Key differences	14
Advantages of Composite Pattern	14
Advantages:	14
Disadvantages of Composite Pattern	15
Disadvantages	15
Conclusion	15
8.1. Summary of the Pattern	15
8.2. Why it was chosen	15
References	16
Books	16
Websites	16

Introduction To Composite Design Pattern

1.1 Overview of Composite design patterns:

The Composite Design Pattern is a structural design pattern that allows developers to compose objects into tree-like structures to represent part-whole hierarchies.

It enables clients to treat individual objects and compositions of objects in a uniform way without needing to distinguish between them.

This pattern is especially useful in systems where objects can contain other objects recursively, forming hierarchical structures.

As part of structural design patterns, the Composite Pattern simplifies the handling of complex object structures by providing a unified interface for both simple and complex elements.

1.2 Intent of the pattern:

The main intent of the Composite Design Pattern is to:

- Treat individual objects and groups of objects uniformly
- Represent hierarchical tree structures efficiently
- Allow recursive composition of objects
- Simplify client code by removing conditional logic

By using this pattern, the system becomes more flexible, scalable, and easier to maintain.

1.3 Key idea (step-by-step construction):

- The core idea of the Composite Pattern is to build a tree structure where:
- Define a common interface for all objects
- Create leaf objects that represent individual elements
- Create composite objects that contain multiple child objects

- Allow composites to contain both leaf and other composite objects
- Perform operations recursively across the structure
- This allows complex structures to be treated as a single unified object.

Problem Analysis & Trade-offs

2.1 Issues in object creation:

In modern software development, creating objects may seem simple at first, but it can become problematic when the objects are complex or require many parameters.

Traditional object creation techniques often lead to code that is hard to read, maintain, and extend.

These challenges result in several common issues, which are discussed below.

2.1.1 Poor object creation design:

Poor object creation design occurs when complex objects are initialized using traditional constructors or setters without a clear and structured approach.

This often leads to difficulties when managing objects that have many attributes, especially when some of these attributes are optional. As a result, the code becomes less organized and harder to understand, maintain, and modify.

For example, initializing an object with many parameters directly inside a constructor can make the code confusing and reduce clarity about the role of each value.

2.1.2 Constructors with many parameters:

Constructors with many parameters occur when a class requires a large number of values to create an object.

This makes it difficult for developers to remember the correct order, type, and meaning of each parameter. As a result, the probability of errors increases, such as passing values in the wrong order or using incorrect data.

In addition, optional parameters make the situation more complicated, because developers may be forced to pass unnecessary or placeholder values just to satisfy the constructor requirements.

2.1.3 Inconsistent object state:

Inconsistent object state occurs when an object is created but not all of its required attributes are properly initialized.

This often happens when setters are used after object creation, allowing the object to exist in a partially valid or invalid state for a period of time.

Such situations can lead to unexpected behavior, runtime errors, and difficulties in debugging, as the object may be used before it is fully ready.

2.1.4 Lack of flexibility:

Lack of flexibility occurs when changes in the object structure require modifications in multiple parts of the codebase.

For example, if new attributes are added to a class, all existing constructors and related code must be updated accordingly.

This makes the system more difficult to maintain and increases the risk of introducing bugs during modifications.

.

2.2 Why traditional approaches fail:

Traditional approaches fail because they rely on:

- Manual traversal of nested structures
- Multiple conditional checks (is product / is box)
- Complex loops for each hierarchy level
- Tight coupling between classes

This results in:

- Poor scalability
- Difficult maintenance
- Repetitive code
- High complexity when hierarchy grows

2.3 Possible design patterns to solve these issues:

2.3.1 Composite Design Pattern

The Composite Pattern solves the problem by:

- Treating individual objects and composite objects uniformly
- Using recursion instead of manual loops
- Allowing dynamic tree structures

.

2.3.2 Flyweight Design Pattern

The Flyweight Pattern reduces memory usage by sharing common object data between multiple objects, but it does not handle hierarchical structures.

Pattern Selection & Justification

3.1 Why Composite Design Pattern is Needed:

The Composite Design Pattern is needed to simplify the handling of such tree-like structures by allowing individual objects and compositions of objects to be treated uniformly. It provides a clear structure where both simple elements (leaf nodes) and complex containers (composite nodes) share the same interface, enabling consistent operations across the entire hierarchy.

This pattern is particularly useful when:

- Objects naturally form a part-whole hierarchy (e.g., folders and files, menus and menu items)
- The system needs to treat single objects and groups of objects in the same way
- Recursive operations over tree structures are required
- Code maintainability and flexibility are important

By using the Composite pattern, developers can avoid complex conditional logic that distinguishes between individual and grouped objects, reduce code duplication, and make it easier to extend the system with new types of components while keeping the overall design

clean and scalable.

3.2 Real-World Analogy:

A real-world example is a warehouse delivery system:

- Small items → Products
- Boxes → Containers
- Large boxes → Containers of containers

All are treated as “delivery items” regardless of their structure level.

3.3 When to Use the Composite Pattern:

The Composite Pattern should be used when:

- Objects form tree structures
- Objects contain other objects recursively
- Clients should treat all objects uniformly
- The structure is dynamic and expandable

Case Study: Online Shopping Delivery System

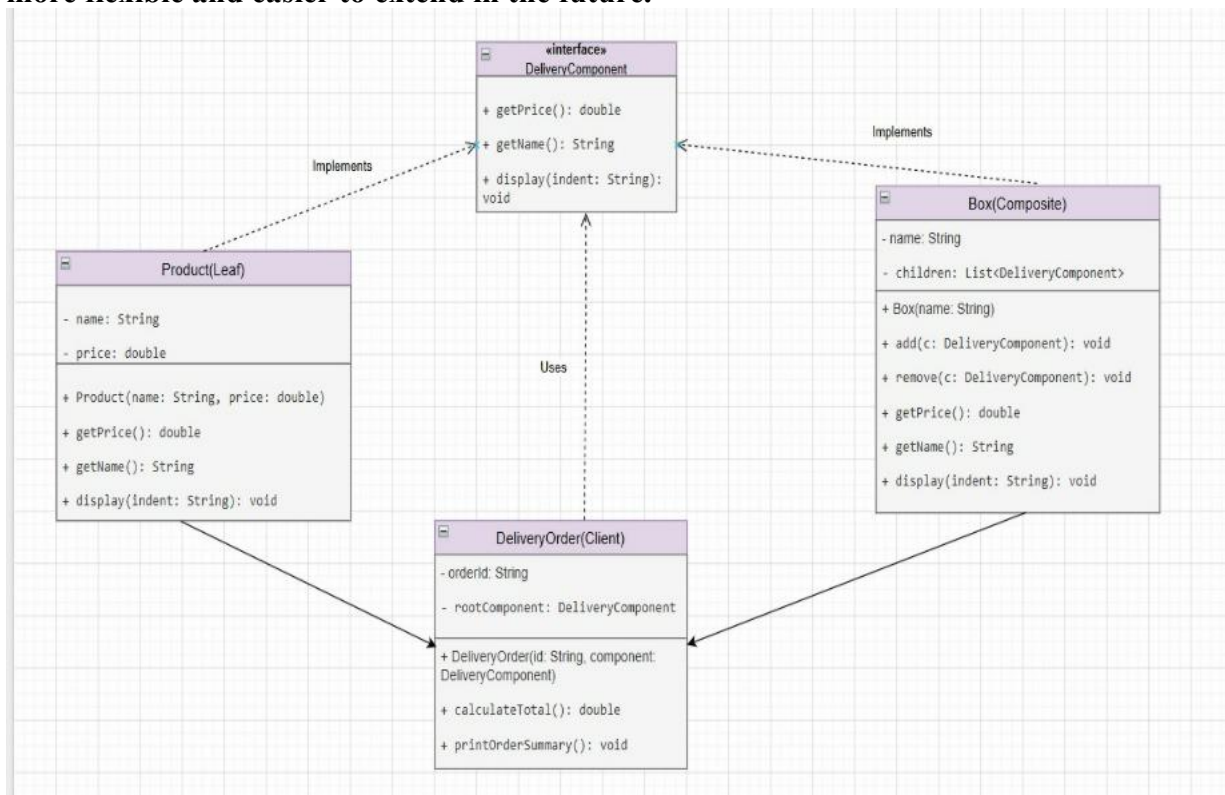
4.1 Problem in Online Shopping Delivery System:

In an online shopping delivery system, products are packed inside boxes before shipment. A box can contain multiple products as well as smaller boxes, and these smaller boxes may also contain products or even more nested boxes. The system must calculate the total price of all products stored inside the main box regardless of how deeply the products are nested. The problem is that handling products and boxes separately makes the code complex and difficult to maintain because the system needs repeated checks and recursive calculations for every inner box. As the structure grows larger, the implementation becomes harder to extend, and adding new container types or changing the pricing logic may require

modifying many parts of the code.

4.2 Applying Composite Pattern as a Solution:

The Composite Pattern provides an effective solution for this problem by allowing both products and boxes to be treated in the same way through a common interface. Each product can directly return its own price, while each box calculates its total price by collecting the prices of all products and smaller boxes contained inside it. Since every object follows the same structure, the system can recursively calculate the final price of the main box without needing complicated conditional statements. This pattern simplifies the design, reduces code duplication, improves maintainability, and makes the delivery system more flexible and easier to extend in the future.



The System consists of :

1. A `DeliveryComponent` interface that defines the common operations for all delivery objects, such as calculating the price, getting the name, and displaying the structure.
2. A `Product` class representing the Leaf object, which stores the product name and price and directly returns its own price.
3. A `Box` class representing the Composite object, which can contain multiple products and smaller boxes, and calculates the total price by summing the prices of

all child components.

4. A DeliveryOrder class representing the Client, which uses the root delivery component to calculate the total order price and print the complete delivery structure.

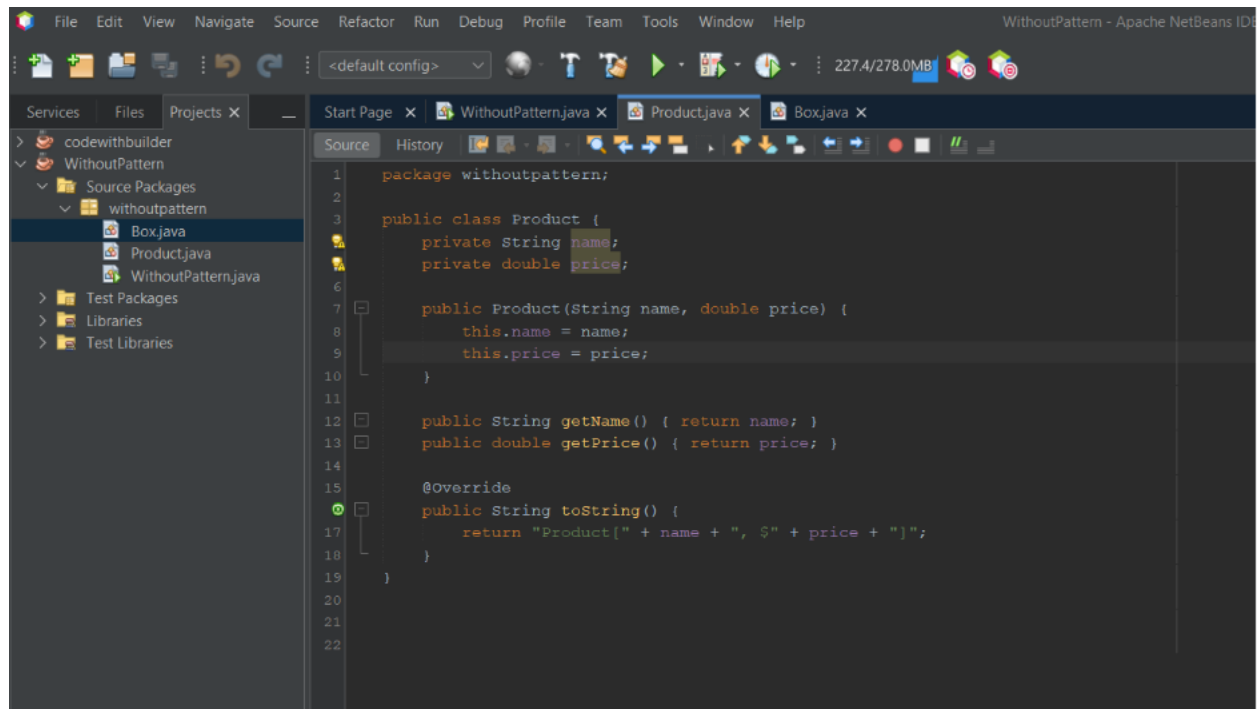
Applying the Composite pattern to Online Shopping Delivery System provides several advantages:

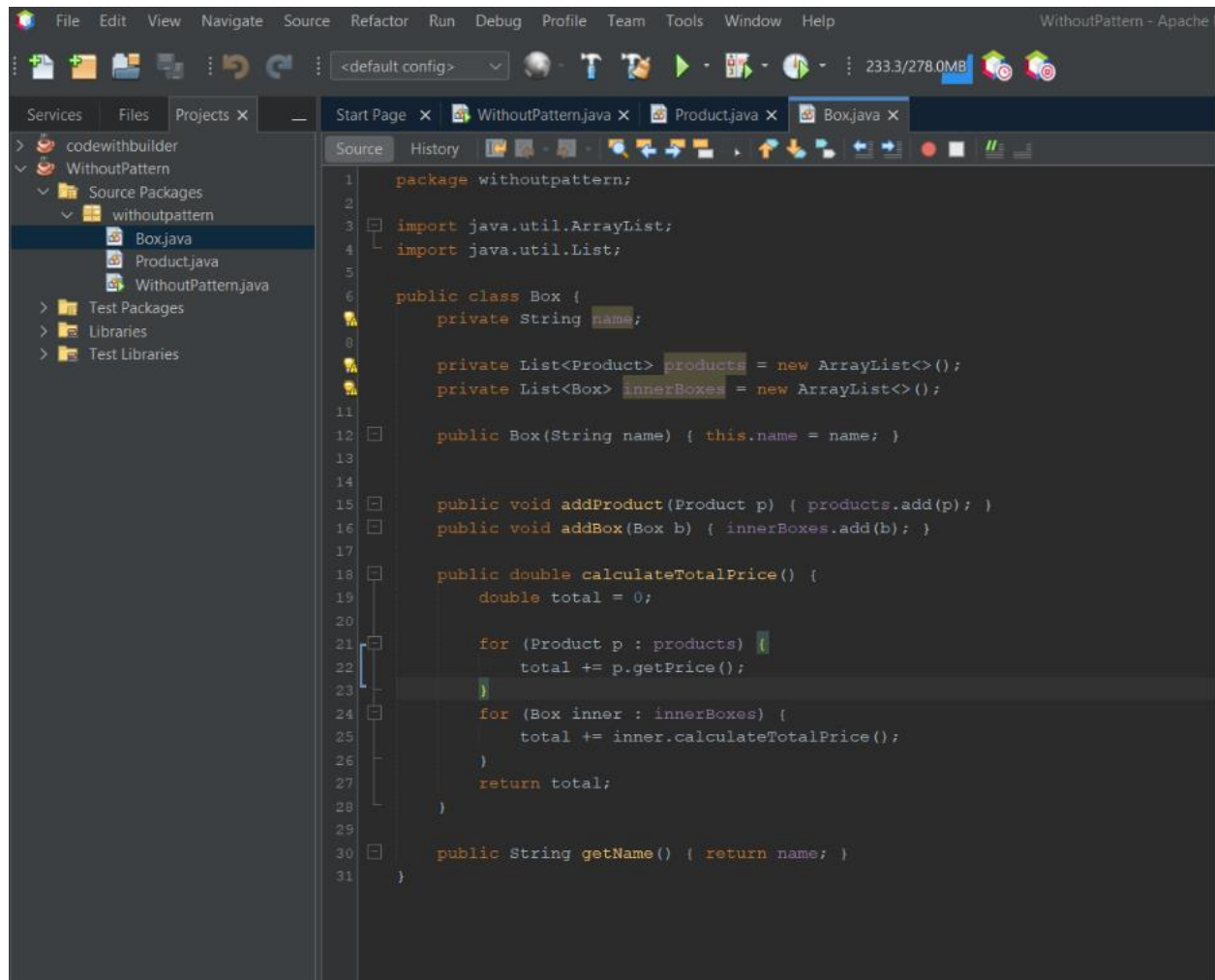
- **Uniform treatment of objects (Box & Product)**
- **Easy recursive price calculation**
- **Scalability (supports unlimited nesting levels)**
- **Flexible and extensible structure**
- **Simplifies client code**

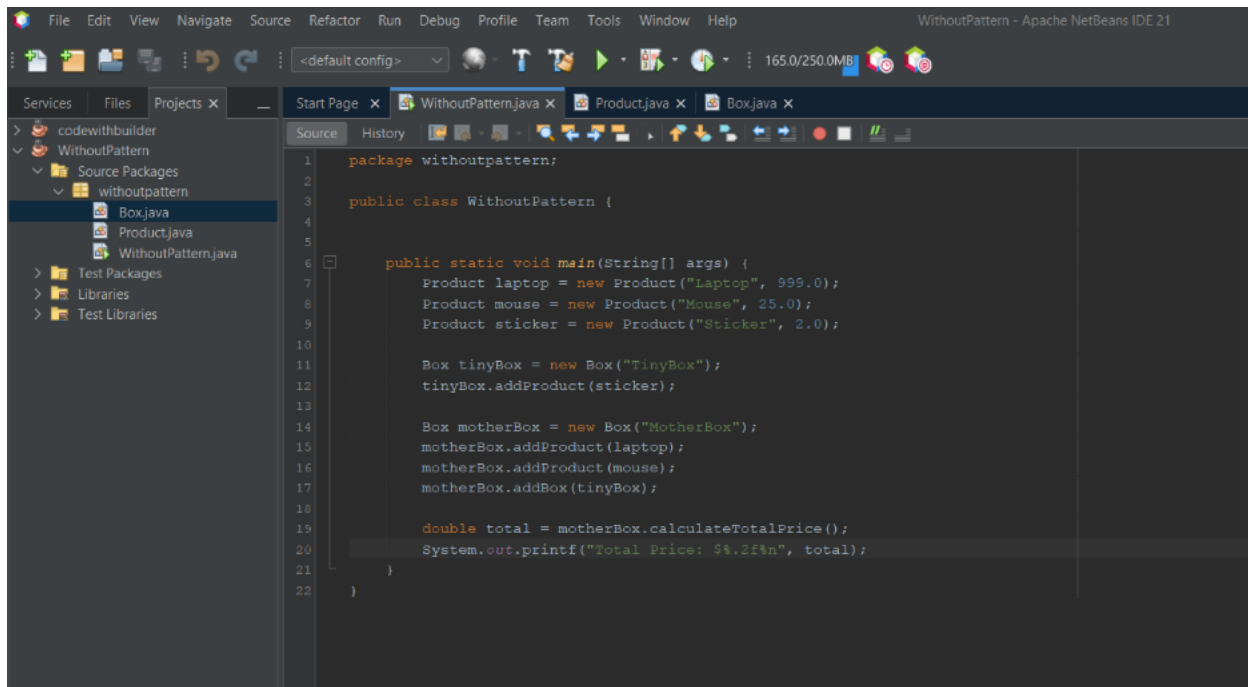
Pattern Implementation

5.1 Code without Composite:

In a traditional implementation of an online shopping delivery system, boxes and products are treated as two completely separate entities.







A product is a simple object that contains basic information such as price, while a box is a more complex object that contains collections of products and possibly other smaller boxes. Since there is no unified structure between them, the system handles each type differently and explicitly.

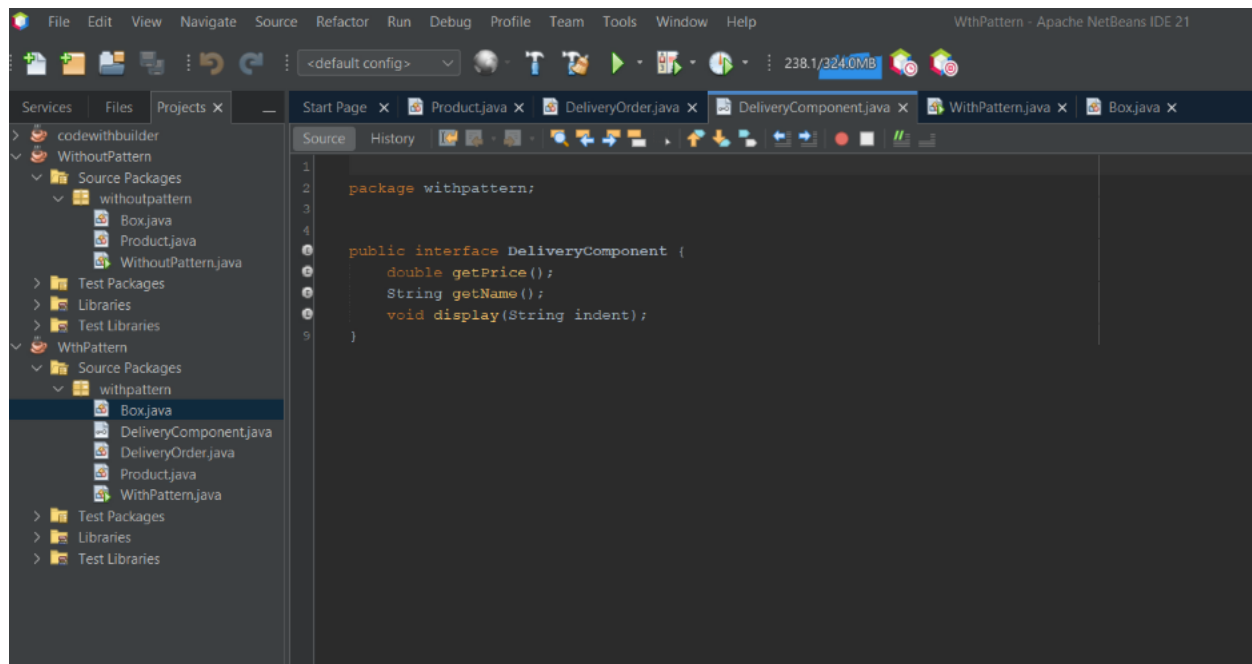
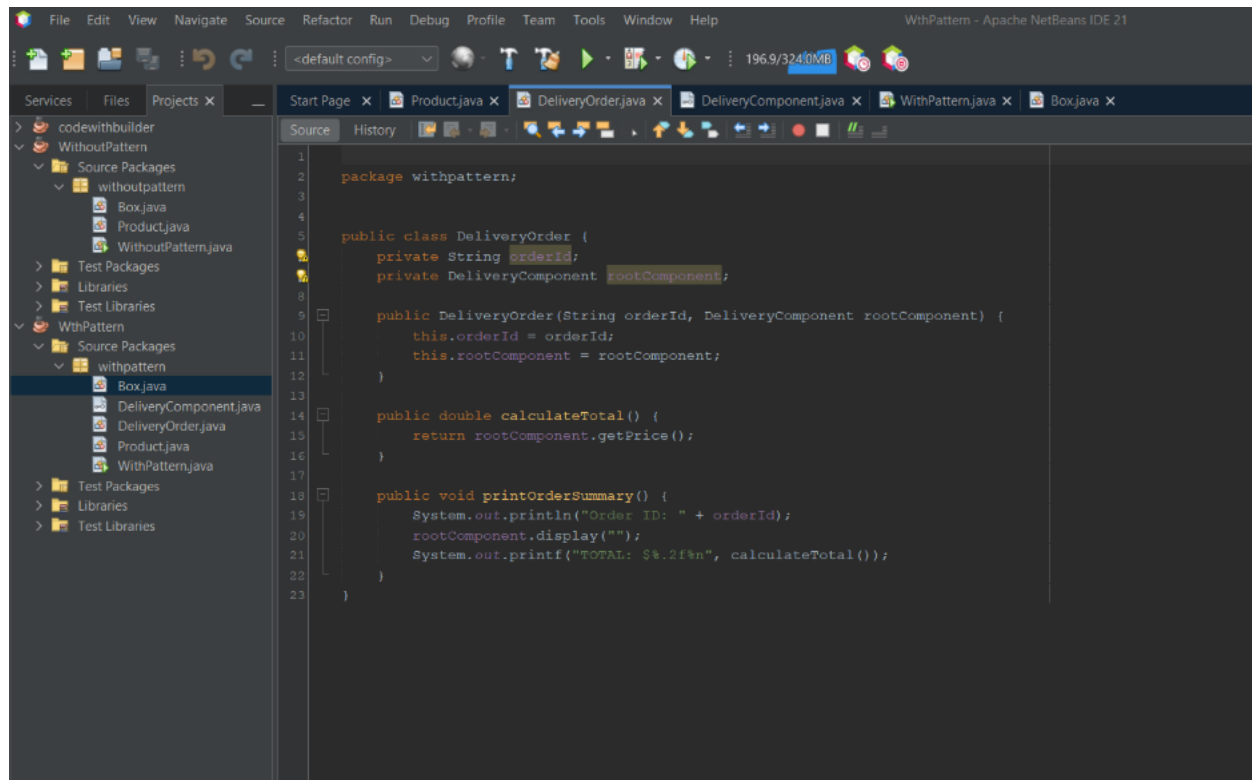
To calculate the total price of a “mother box,” the system manually traverses its contents. It first iterates through all direct products inside the box and sums their prices. Then, for each smaller box inside it, the system recursively applies the same logic, going deeper until it reaches the innermost level where only products exist. This process relies heavily on recursion and nested loops to navigate the hierarchical structure.

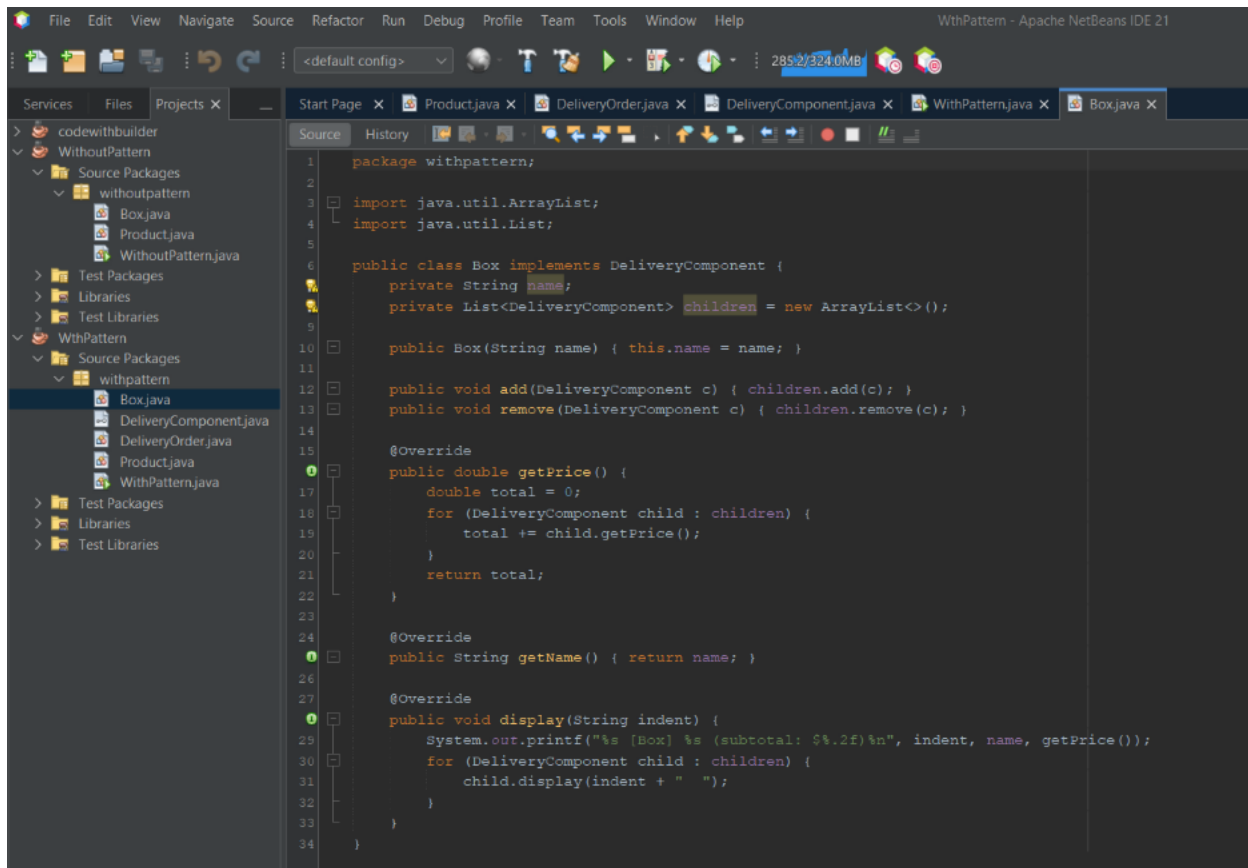
This approach works, but it tightly couples the calculation logic with the structure of the boxes and products. Every time a new type of container is introduced, the logic must be manually updated. As a result, the system becomes harder to maintain and less flexible as complexity increases.

5.2 Code with Composite:

To overcome the limitations of the traditional approach, the Composite Design Pattern provides a more structured and scalable way to represent hierarchical relationships

between products and boxes:





In this approach, both products and boxes are treated under a common abstraction, allowing them to be handled uniformly. A single interface or abstract class defines a common behavior, such as calculating the total price, which is implemented differently depending on whether the object is a simple product or a composite box.

This design enables each box to contain a collection of components, where each component can be either a product or another box. The key idea is that the system treats individual objects and compositions in the same way, which simplifies the process of navigating through the structure.

The total price calculation becomes a clean recursive process, where each component is asked to return its own cost. If the component is a product, it returns its price directly; if it is a box, it delegates the calculation to its children and aggregates the results. This approach improves flexibility, enhances code readability, and makes the system much easier to extend without modifying existing logic.

5.3 Key differences:

The primary difference between the traditional programming approach and the Composite Pattern is the way objects are handled within the system. In the traditional approach, products and boxes are managed using separate logic and multiple conditional statements. The system repeatedly checks whether the object is a product or a box, and if it is a box, it must manually iterate through all nested elements to calculate the total price. As the system grows and more nested boxes are added, the code becomes more complex, harder to maintain, and difficult to extend. In contrast, the Composite Pattern treats both products and boxes through the same common interface, allowing the system to interact with all objects uniformly. Each object becomes responsible for handling its own calculation, making the code cleaner, more organized, reusable, and easier to scale without modifying the existing structure.

Advantages of Composite Pattern

Advantages:

The Composite Design Pattern offers several key advantages that make it highly suitable for systems that manage hierarchical and nested structures

1. Uniform Treatment of Objects

Products and boxes can be handled using the same interface, which simplifies the system design.

2. Simplified Client Code

The client does not need to distinguish between single objects and complex object groups.

3. Improved Maintainability

The code becomes easier to read, organize, and maintain because responsibilities are clearly separated.

4. Easy Extensibility

New component types can be added without modifying existing code structure.

5. Supports Recursive Structures

The pattern efficiently handles nested and tree-like structures such as boxes within boxes.

Disadvantages of Composite Pattern

Disadvantages:

1. Increased System Complexity

The recursive structure of the pattern may make the system harder to understand and debug.

2. Reduced Type Safety

Using a common interface for all components may allow unsupported operations on certain objects.

3. Harder Initial Design

Designing the component hierarchy and recursive relationships requires careful planning.

Conclusion

8.1 Summary of the pattern:

The Composite Design Pattern is a structural design pattern that focuses on organizing objects into hierarchical tree-like structures.

It allows individual objects and groups of objects to be treated uniformly through a common interface. By representing simple elements as leaf objects and container elements as composite objects, the pattern simplifies the management of nested structures and recursive operations.

This structured approach makes the Composite pattern highly effective for developing flexible, maintainable, and scalable systems, especially in applications that involve complex hierarchical relationships such as online shopping delivery systems.

8.2 Why it was chosen:

The Composite Design Pattern was chosen for the online shopping delivery system due to the hierarchical nature of the delivery structure.

In the system, a box can contain multiple products as well as smaller boxes, and these smaller boxes may also contain additional products or nested boxes. Using traditional programming approaches in such scenarios leads to complex conditional logic and repetitive recursive calculations.

By applying the Composite pattern, both products and boxes can be treated uniformly through a common interface, allowing the system to calculate prices and manage components in a simpler and more organized way.

This approach not only simplifies the implementation of recursive operations but also improves code readability, maintainability, and scalability.

Additionally, the pattern supports future extensibility, making it easier to introduce new delivery component types or additional features without requiring major modifications to the existing system.

References

Books

1. Design Patterns: Elements of Reusable Object-Oriented Software
Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).
Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Websites

1. Refactoring Guru
Composite Design Pattern.
<https://refactoring.guru/design-patterns/composite>
2. GeeksforGeeks
Composite Design Pattern in Java.
<https://www.geeksforgeeks.org/composite-design-pattern/>

